

Market BF

Marketplace locale et gestion commerciale
pour les commerces du Burkina Faso

DOSSIER DE SOUTENANCE

Application web full-stack — Node.js / Express • React • SQLite (Turso)

Déployée en production : <https://marketbf.onrender.com>

Présenté par : NITIEMA Martial • [Nom du binôme]

Juillet 2026

Sommaire

1. Introduction et contexte
2. Objectifs du projet
3. Acteurs et fonctionnalités réalisées
4. Stack technique
5. Architecture de l'application
6. Modèle de données
7. Commandes et paiements
8. Sécurité
9. Déploiement et mise en production
10. Démonstration et déroulé de la soutenance
11. Difficultés rencontrées et solutions
12. Perspectives d'évolution
13. Conclusion

1. Introduction et contexte

Au Burkina Faso, le commerce de proximité constitue le cœur de l'activité économique. Pourtant, la grande majorité des commerçants gèrent encore leurs stocks, leurs ventes et leurs commandes de façon manuelle, sans aucun outil numérique. Côté clients, trouver un produit précis suppose de se déplacer de boutique en boutique, sans garantie de disponibilité ni visibilité sur les prix.

Market BF répond à ce double problème : offrir aux commerçants burkinabè un outil complet de gestion commerciale (produits, stocks, commandes, statistiques) et offrir aux clients une marketplace géolocalisée permettant de rechercher des produits, vérifier leur disponibilité en temps réel, commander et payer selon les moyens de paiement locaux (Orange Money, Moov Money, Wave, paiement à la livraison).

Le résultat est une application web full-stack opérationnelle, déployée en production à l'adresse <https://marketbf.onrender.com>, comptant environ **7 000 lignes de code**, **9 modules d'API REST** et **16 pages** d'interface réparties sur trois espaces (client, commerçant, administrateur).

2. Objectifs du projet

Le projet poursuit cinq objectifs, tous atteints dans la version livrée :

- Digitaliser la gestion des commerces locaux (produits, stocks, ventes, commandes)
- Faciliter la recherche de produits pour les clients, avec disponibilité en temps réel
- Permettre la gestion des stocks en temps réel, avec alertes et historique des mouvements
- Intégrer un système de commande et de paiement adapté au contexte burkinabè
- Améliorer la visibilité des commerces grâce à la géolocalisation sur carte interactive

3. Acteurs et fonctionnalités réalisées

La plateforme distingue trois rôles, chacun disposant de son propre espace et de droits contrôlés côté serveur.

3.1. Espace client

- Inscription / connexion (e-mail + mot de passe, ou connexion Google en un clic)
- Mot de passe oublié : lien de réinitialisation envoyé par e-mail (valable 1 heure)
- Catalogue de produits avec recherche par nom et filtrage par catégorie
- Carte interactive des boutiques (Leaflet) avec géolocalisation
- Fiche boutique : produits, disponibilité en temps réel, avis clients
- Panier et passage de commande avec choix du mode de paiement
- Reçu envoyé par e-mail à la commande, confirmation par e-mail à la livraison
- Historique et suivi de l'état des commandes
- Dépôt d'avis (note 1-5 + commentaire) sur les boutiques

3.2. Espace commerçant

- Création de la boutique avec localisation sur carte ; visible après validation par l'admin
- Ajout, modification et suppression des produits, avec upload de photos
- Gestion du stock : quantités, seuils d'alerte, mouvements d'entrée/sortie tracés
- Alertes automatiques de stock faible
- Réception des commandes et mise à jour de leur statut (en attente → confirmée → livrée / annulée)

- Statistiques de vente : chiffre d'affaires, volume, produits les plus vendus (graphiques Recharts)

3.3. Espace administrateur

- Validation ou rejet des boutiques en attente
- Gestion des utilisateurs (liste, suppression)
- Statistiques globales de la plateforme

4. Stack technique

Les technologies ont été choisies avec un fil conducteur : une plateforme réellement utilisable au Burkina Faso — accessible depuis n'importe quel navigateur (téléphone ou PC, sans installation) et avec un coût d'infrastructure nul.

Couche	Technologies
Backend	Node.js, Express, better-sqlite3 / libSQL (Turso), jsonwebtoken, bcryptjs, express-validator, multer, nodemailer
Frontend	React 18, Vite, React Router, Tailwind CSS, Axios, Leaflet / React-Leaflet, Recharts, Lucide React, React Hot Toast
Infrastructure	Docker & Docker Compose, Nginx, Render (hébergement), Turso (base de données), Cloudinary (images), GitHub (déploiement continu)

Choix	Motivation
Application web responsive (React)	Accessible immédiatement sur tout téléphone ou ordinateur via un simple navigateur, sans installation ni store d'applications.
Node.js + Express	Écosystème JavaScript unifié entre le frontend et le backend, léger et rapide à déployer.
SQLite en local, Turso (libSQL) en production	Zéro administration, transactions ACID, hébergement gratuit et répliqué ; les données survivent aux redémarrages du serveur.
JWT (HS256) + Google Identity Services	Authentification stateless standard, complétée par une connexion Google en un clic.
Leaflet + OpenStreetMap	Cartographie gratuite et open source, sans clé d'API ni carte bancaire, bonne couverture du Burkina Faso.

5. Architecture de l'application

L'application suit une architecture **client-serveur découplée** : une API REST stateless côté serveur, consommée par une single-page application React. En production, un conteneur Docker unique sert à la fois l'API et le build statique du frontend (même origine, pas de problème CORS).

Composant	Rôle
Frontend React (SPA)	16 pages réparties en trois espaces : client (7), commerçant (6), administrateur (3). Routage protégé par rôle via un contexte d'authentification global.
API REST Express	9 modules de routes : auth, shops, products, stock, orders, reviews, stats, admin, upload. Toutes les routes préfixées par /api, protégées par JWT.
Middleware d'authentification	Vérification du token JWT et contrôle des rôles (client / merchant / admin) avant chaque route protégée.
Base de données	SQLite locale en développement ; Turso (libSQL) en production. Schéma créé et peuplé automatiquement au premier démarrage.
Stockage d'images	Upload via multer ; fichiers sur disque en local, sur Cloudinary en production (persistance garantie).
E-mails transactionnels	Nodemailer + SMTP : reçu de commande, confirmation de livraison, lien de réinitialisation de mot de passe.

Vue d'ensemble des endpoints

Ressource	Endpoints principaux
auth	POST /register, /login, /google, /forgot-password, /reset-password — GET /me, /config
shops	GET /, /:id, /merchant/mine — POST / — PUT /:id
products	GET /, /:id, /merchant/mine — POST / — PUT /:id — DELETE /:id
stock	GET /product/:id, /movements/:id, /alerts — PUT /product/:id — POST /movement
orders	POST / — GET /, /:id — PUT /:id/status
reviews	GET /shop/:shopId — POST / — DELETE /:id
stats	GET /overview, /sales, /products
admin	GET /shops, /users, /stats — PUT /shops/:id/validate — DELETE /users/:id

6. Modèle de données

Le schéma relationnel compte neuf tables. Il est créé et peuplé automatiquement au premier démarrage avec des boutiques et produits de démonstration burkinabè.

Table	Contenu
users	Clients, commerçants et admins ; rôle, e-mail unique, mot de passe haché (bcrypt)
shops	Boutiques géolocalisées (latitude/longitude), statut pending / active / rejected
products	Produits rattachés à une boutique : nom, catégorie, prix, image
stock	Quantité disponible et seuil d'alerte par produit
stock_movements	Historique horodaté des entrées / sorties de stock
orders	Commandes : client, boutique, statut, montant, adresse de livraison
order_items	Lignes de commande : produit, quantité, prix unitaire
payments	Paieement lié à une commande : méthode, montant, statut
reviews	Avis client sur une boutique : note 1 à 5 + commentaire

7. Commandes et paiements

La commande est traitée de façon **transactionnelle** : le stock de chaque produit est décrémenté atomiquement (protection contre les commandes simultanées — pas de survente possible) et restitué automatiquement si la commande est annulée.

Quatre méthodes de paiement sont proposées : **Orange Money, Moov Money, Wave et paiement à la livraison**. Chaque commande génère un enregistrement de paiement avec méthode, montant et statut (pending → completed à la livraison, failed en cas d'annulation) : la plateforme conserve donc un historique complet des transactions. Les paiements mobiles sont **simulés** : l'intégration réelle des API Orange Money / Moov / Wave nécessite un contrat marchand et un agrégateur agréé. Le flux complet étant déjà implémenté, brancher un agrégateur réel ne demandera aucune refonte. Un reçu détaillé est envoyé par e-mail au client à chaque commande.

8. Sécurité

Domaine	Mise en œuvre dans Market BF
Authentification	JWT signé HS256 avec secret fort obligatoire en production (le serveur refuse de démarrer sinon) ; mots de passe bcrypt 12 rounds ; politique de complexité ; anti-énumération de comptes (réponses en temps constant).
Données sensibles	Mots de passe hachés bcrypt ; tokens de réinitialisation aléatoires 256 bits stockés hachés SHA-256, usage unique, expiration 1 h ; HTTPS de bout en bout en production.
Paieements	Enregistrement transactionnel du paiement ; transitions de statut contrôlées côté serveur selon le rôle.
Protection de l'API	Contrôle des rôles sur chaque route ; Helmet (en-têtes HTTP) ; rate limiting global (300 req/15 min) et renforcé sur l'authentification (20 tentatives/15 min) ; CORS restreint ; validation systématique des entrées (express-validator) ; uploads durcis (extension + type MIME, 5 Mo max, nom aléatoire) ; payload JSON limité à 100 ko.

Domaine	Mise en œuvre dans Market BF
Persistance des données	Base hébergée sur Turso (répliquée, persistante, indépendante du serveur applicatif) ; images sur Cloudinary. Un redéploiement ou un crash ne perd aucune donnée.

9. Déploiement et mise en production

L'application est **conteneurisée avec Docker** (build multi-étapes : build Vite du frontend, puis image Node servant l'API et les fichiers statiques) et déployée sur **Render** via un blueprint `render.yaml`. Chaque *git push* sur GitHub déclenche automatiquement un redéploiement.

- Hébergement : Render (plan gratuit), un seul service tout-en-un — <https://marketbf.onrender.com>
- Base de données : Turso (SQLite hébergé, libSQL) — les données survivent aux redémarrages
- Images : Cloudinary — stockage persistant des photos produits et logos
- Secrets (JWT, SMTP, OAuth Google) : variables d'environnement Render, jamais dans le code
- Développement local : `./start.sh` ou `docker-compose up`, base SQLite locale automatique
- Coût total de l'infrastructure : 0 F CFA

10. Démonstration et déroulé de la soutenance

La soutenance dure **10 minutes** et est présentée en binôme :

Partie	Intervenant	Contenu	Durée
Partie 1	Intervenant 1	Contexte et problématique, présentation de Market BF, fonctionnalités des trois espaces, parcours d'une commande	≈ 5 min
Partie 2	Intervenant 2	Architecture et choix techniques, sécurité, déploiement, démonstration en direct, conclusion	≈ 5 min

L'application est accessible en ligne : **<https://marketbf.onrender.com>** (ouvrir le site 2 minutes avant la soutenance pour réveiller le service gratuit). Comptes de démonstration :

Rôle	E-mail	Mot de passe
Administrateur	<code>admin@marketbf.com</code>	Admin123!
Commerçant	<code>merchant1@marketbf.com</code>	Merchant1!
Client	<code>client@marketbf.com</code>	Client123!

Scénario de démonstration : (1) le client recherche un produit, consulte la carte, commande avec Orange Money et reçoit son reçu par e-mail ; (2) le commerçant voit la commande, la confirme puis la marque livrée — le stock a été décrémenté et une alerte apparaît s'il passe sous le seuil ; (3) l'administrateur valide une nouvelle boutique en attente.

11. Difficultés rencontrées et solutions

Difficulté	Solution apportée
Persistance des données sur un hébergement gratuit (disque effacé à chaque redéploiement Render)	Externalisation de la base vers Turso (SQLite hébergé) et des images vers Cloudinary.
Cohérence du stock lors de commandes simultanées	Commandes transactionnelles avec décrément atomique et restitution du stock en cas d'annulation.
Intégration des paiements mobiles sans contrat marchand	Flux de paiement complet mais simulé, architecture prête pour un agrégateur réel.

Difficulté	Solution apportée
Coût des services de cartographie	Leaflet + OpenStreetMap : gratuit, sans clé API, bonne couverture du Burkina Faso.
Sécurité d'une application exposée publiquement	Durcissement systématique : Helmet, rate limiting, validation des entrées, uploads contrôlés, secrets en variables d'environnement.

12. Perspectives d'évolution

- Intégration réelle des paiements mobiles via un agrégateur (Orange Money, Moov, Wave) puis carte bancaire
- Génération de factures PDF téléchargeables
- Module de livraison intégré (suivi livreur, frais calculés par distance)
- Recommandations de produits par intelligence artificielle
- Application mobile (React Native ou PWA installable) pour capitaliser sur l'existant
- Wallet numérique et programme de fidélité
- Montée en charge : migration PostgreSQL, multi-boutiques par commerçant

13. Conclusion

Market BF démontre qu'une plateforme complète de digitalisation du commerce local est réalisable avec des technologies open source et un coût d'infrastructure nul. Les trois rôles disposent chacun d'un espace fonctionnel, et la chaîne complète — de la recherche géolocalisée d'un produit jusqu'à sa livraison confirmée par e-mail — fonctionne en production, avec des données persistantes et un socle de sécurité sérieux.

Le projet constitue une base saine pour les évolutions prévues : paiements réels, livraison intégrée et application mobile. Il illustre la maîtrise du cycle complet d'un projet web : analyse du besoin, choix d'architecture argumentés, développement full-stack, sécurisation et mise en production.